DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 1: INTRODUCTION

Question Bank with Comprehensive Answers

SYLLABUS OVERVIEW — Unit 1: Introduction

Subject Code: CT 652 | Lectures: 3 hrs/week | Tutorial: 1 hr/week | Practical: 3 hrs/week | Full Marks: 80 | Pass Marks: 32

Unit	Topic	Hours	Marks Dist.
1	Introduction	3 hours	4
2	Data Models	7 hours	12
3	Relational Languages and Relational Model	7 hours	12
4	Database Constraints and Normalization	6 hours	12
5	Query Processing and Optimization	4 hours	8
6	File Structure and Hashing	4 hours	8
7	Transactions Processing and Concurrency Control	6 hours	12
8	Crash Recovery	4 hours	6
9	Advanced Database Concepts	4 hours	6
Total		**45 hours**	**80**

Chapter 1 Topics (3 Hours | 4 Marks)

- 1.1 Concepts and Applications of Database
- 1.2 Objective and Evolution of DBMS
- 1.3 Data Abstraction and Data Independence
- 1.4 Schema and Instances
- 1.5 Concepts of DDL, DML and DCL

Course Objective: To provide fundamental concepts, theory and practices in design and implementation of Database Management Systems.

CHAPTER 1: INTRODUCTION — Questions & Answers

Q: Briefly explain the three levels of data abstraction in RDBMS. What is physical data independence?
Asked: 2082 Kartik (Back Exam) | Marks: [3+1]

Answer

Three Levels of Data Abstraction

Data abstraction refers to hiding the complexity of the actual data storage and representation from users. A Database Management System (DBMS) provides three levels of abstraction, also known as the ANSI/SPARC three-level architecture:

1. Physical Level (Internal Level)

This is the lowest level of data abstraction. It describes HOW data is actually stored in the database — the actual data structures, storage devices, file organization, indexing methods, and access paths. Database administrators (DBAs) work at this level. Users at this level deal with complex low-level data structures.

- Deals with: physical storage (hard disk, SSD), file structures, B+ trees, hash indexes
- Concerned with: efficiency, storage optimization, access speed
- Example: Data stored as records on disk using B+ tree index on EmployeeID

2. Logical Level (Conceptual Level)

This is the middle level of data abstraction. It describes WHAT data is stored in the database and the relationships among those data. Programmers and database administrators work at this level. It defines the overall structure of the entire database.

- Deals with: tables, attributes, relationships, constraints, integrity rules
- Concerned with: what entities exist, what relationships hold
- Example: Employee(empid, name, salary, dept_id) related to Department(dept_id, dname)

3. View Level (External Level)

This is the highest level of abstraction. It describes only a PART of the entire database relevant to a particular user or group of users. Different users see different views of the database. This level provides security by hiding certain data from specific users.

- Deals with: customized views for individual users
- Concerned with: data security, user convenience
- Example: A bank teller sees only customer account details, not loan officer data

```
           [ View 1 ]     [ View 2 ]     [ View 3 ]     <- External Level (Users)
                     External/View Level
           ____________________________________________
           [        Conceptual/Logical Level       ]   <- What data (tables,
relations)
           ____________________________________________
```

```
[        Physical/Internal Level        ]   <- How data is stored
```

Physical Data Independence

Physical data independence is the ability to modify the physical schema (internal level) without affecting the conceptual schema or the user views (external level). In other words, changes in how data is physically stored should NOT require changes in application programs or user queries.

Examples of Physical Data Independence:

- Changing storage organization from sequential files to B+ tree index should not affect SQL queries
- Moving the database from HDD to SSD without changing application code
- Adding or removing an index on a column without modifying the conceptual schema
- Changing the block size used for data storage

Physical data independence is generally easier to achieve than logical data independence, and most modern DBMSs provide it. It is one of the primary advantages of DBMS over traditional file systems.

Q: Why do we prefer DBMS over file system for handling databases? Briefly explain.
Asked: 2081 Chaitra (Regular Exam) | Marks: [4]

Answer

Advantages of DBMS over File System

Traditional file systems store data in separate files for different applications, leading to numerous problems. A Database Management System (DBMS) overcomes these limitations. Key reasons to prefer DBMS:

1. Elimination of Data Redundancy and Inconsistency

In file systems, the same data is often duplicated across multiple files (e.g., a student's name in both the examination file and the library file). This leads to data inconsistency when one copy is updated but others are not. DBMS stores data in a central, integrated manner, minimizing redundancy and ensuring consistency through normalization.

2. Data Sharing and Concurrent Access

In file systems, data files are typically owned by specific programs. DBMS allows multiple users and applications to access the same data simultaneously with proper concurrency control mechanisms (locking, transactions), ensuring data integrity.

3. Data Independence

DBMS provides both physical and logical data independence. Physical changes (new storage hardware, different indexing) do not affect application programs. Logical changes (adding new attributes) do not affect other applications using the database.

4. Efficient Data Access

DBMS uses query processing and optimization techniques (B+ trees, hashing, query optimizer) to retrieve data efficiently. The DBMS automatically finds the most efficient way to execute queries, unlike file systems where the programmer must explicitly code file traversal logic.

5. Data Integrity and Security

DBMS enforces integrity constraints (primary keys, foreign keys, check constraints) at the database level, ensuring data correctness. It also provides granular access control — different users can be granted different privileges on different tables/views.

6. Backup and Recovery

DBMS provides built-in mechanisms for backup and crash recovery (log-based recovery, checkpoints). If a system failure occurs, the DBMS can restore the database to a consistent state. File systems require manual backup procedures.

7. Reduced Application Development Time

DBMS provides a high-level query language (SQL) that simplifies data manipulation. Programmers do not need to write complex file I/O code. This significantly reduces development and maintenance time.

8. Support for Multiple User Interfaces

DBMS supports various interfaces: SQL for programmers, forms-based interfaces for casual users, graphical interfaces for end users, and APIs for application developers.

Q: Define database and database management system. List advantages of DBMS over file management.

Asked: 2079 Chaitra (Regular Exam) | Marks: [1+3]

Answer

Database

A database is an organized, integrated collection of logically related data that represents some aspect of the real world (called a mini-world or Universe of Discourse). A database is designed, built, and populated with data for a specific purpose and a target user group.

Example: A university database stores information about students, courses, instructors, and enrollments. All this data is interrelated and managed together.

Database Management System (DBMS)

A DBMS is a software system (collection of programs) that enables users to create, define, maintain, and control access to a database. It provides an environment that is both convenient and efficient to use. Examples include: MySQL, Oracle, MS SQL Server, PostgreSQL.

The DBMS provides: a data definition language (DDL) to define schemas, a data manipulation language (DML) to insert/query/update/delete data, access control, concurrency management, and recovery facilities.

Advantages of DBMS over File Management System

- Controlled data redundancy: DBMS minimizes duplication through normalization; file systems store the same data in multiple files
- Data consistency: Updating one place updates everywhere; file systems risk inconsistent copies

- Data sharing: Multiple users and applications access data simultaneously with concurrency control
- Data integrity: Constraints (primary key, foreign key, check) enforced automatically
- Data security: Role-based access control; file systems only provide file-level permissions
- Data independence: Physical/logical changes don't break applications
- Efficient data access: Query optimizer picks the best execution plan automatically
- Backup and recovery: Automatic transaction log-based recovery from failures
- Reduced application complexity: SQL replaces complex file I/O code

---

Q: Define database language. Compare the DBMS with file processing system.
Asked: 2079 Jestha (Back Exam) | Marks: [2+3]

Answer

Database Language

A database language is a formal language used to define, manipulate, and control data in a database. The three main types of database languages are:

a) Data Definition Language (DDL)

DDL is used to define and modify the database structure (schema). It specifies the structure of the database, including tables, attributes, data types, and constraints. DDL statements are compiled and stored in the data dictionary (system catalog).

- CREATE TABLE — creates a new table
- ALTER TABLE — modifies an existing table structure
- DROP TABLE — removes a table from the database
- CREATE INDEX — creates an index on a table

Example: CREATE TABLE Student (sid INT PRIMARY KEY, name VARCHAR(50), age INT);

b) Data Manipulation Language (DML)

DML is used to insert, retrieve, modify, and delete data from the database. DML can be procedural (specifies how to get data) or non-procedural/declarative (specifies what data is needed — SQL is declarative).

- SELECT — retrieves data
- INSERT — inserts new records
- UPDATE — modifies existing records
- DELETE — removes records

Example: SELECT name, age FROM Student WHERE age > 20;

c) Data Control Language (DCL)

DCL controls access rights and permissions in the database. It manages user privileges.

- GRANT — gives privileges to a user
- REVOKE — removes privileges from a user

Example: GRANT SELECT ON Student TO user1;

Comparison: DBMS vs File Processing System

Aspect	DBMS	File Processing System
Data Redundancy	Minimized through normalization and central storage	High redundancy; same data stored in multiple files
Data Consistency	Ensured through constraints and single data store	Difficult; inconsistency arises when files are updated separately
Data Independence	Supported (physical and logical)	Not supported; programs depend on file structures
Security	Role-based access control at field/table level	Only file-level OS permissions
Concurrency	Handled via locking and transaction management	No built-in concurrency control
Recovery	Automatic recovery via transaction logs	Manual recovery; prone to data loss
Query Processing	SQL with built-in optimizer	Programmer must write explicit file access code
Integrity	Constraints enforced by DBMS	Integrity checks must be coded in each application
Cost	Higher initial cost (license, hardware)	Lower initial cost, but high long-term maintenance

---

Q: What are the drawbacks of file system to store data? Explain.
Asked: 2072 Magh (Back Exam) | Marks: [4]

Answer

Drawbacks/Limitations of File System

Before DBMS, organizations stored data in file systems. These file-based systems suffered from many serious problems:

1. Data Redundancy

The same data is stored in multiple files for different departments or applications. For example, student names and addresses may be stored in both the examination department file and the library system file. This wastes storage space and leads to maintenance problems.

2. Data Inconsistency

Because data is duplicated across files, when one copy is updated, others may not be updated simultaneously. This creates inconsistent data. For example, if a student's address changes and it is updated in only one file but not the other, the two files will have different addresses for the same student.

3. Difficulty in Accessing Data

Every new type of query requires writing a new program. Conventional file systems do not provide an efficient way to retrieve data that meets certain criteria. There is no general query language — each access requires custom programming.

4. Data Isolation

Data is scattered in various files in different formats (different encodings, field widths, etc.). It is very difficult to write new application programs to retrieve appropriate data scattered across multiple files in different formats.

5. Integrity Problems

Integrity constraints (e.g., balance must be > 0, age must be a positive integer) are buried within application programs rather than being stated explicitly. When new constraints need to be added, all programs must be modified — a maintenance nightmare.

6. Atomicity of Updates

If a system failure occurs during a multi-step update (e.g., transferring money: debit account A, credit account B), it is difficult to ensure either both operations complete or neither does. File systems do not provide transaction atomicity.

7. Concurrent-Access Anomalies

When multiple users access the same file simultaneously without proper controls, anomalies occur. For example, two clerks booking the last airline seat simultaneously might both confirm the booking, leading to double booking.

8. Security Problems

File systems provide security at the file level only (read/write/execute for a whole file). Fine-grained security (allowing user A to see salary but not SSN, user B to see both) is not possible in file systems.

Q: What do you mean by Schema and Instances? Briefly explain the different levels of data abstraction.

Asked: 2079 Ashwin (Back Exam); also 2075 Bhadra [2+2] | Marks: [2+3]

Answer

Schema

A schema is the overall description/design of the database — the logical structure of the database as defined and stored in the data dictionary. It specifies the organization of data: what tables exist, what columns each table has, what data types are used, what constraints apply, and what relationships exist between tables.

- Also called the 'database blueprint' or 'intension' of the database
- Rarely changes once the database is designed
- Example: Student(sid: INT, name: VARCHAR(50), age: INT, dept_id: INT)

There are different types of schemas corresponding to the three levels of abstraction: physical schema (physical level), logical/conceptual schema (logical level), and subschemas or view schemas (view level).

Instances

An instance (also called the database state or database extension) is the actual content of the database at a particular point in time — the collection of all the data values currently stored. The database instance changes every time data is inserted, updated, or deleted.

- Also called the 'current state' or 'extension' of the database
- Changes frequently with every data modification
- Example: {(1, 'Ram Bahadur', 20, 5), (2, 'Sita Sharma', 22, 3)} — actual student records

Aspect	Schema	Instance
Definition	Structure/description of the database	Actual data stored at a given moment
Change Frequency	Rarely changes	Changes with every DML operation
Also Called	Intension / Database blueprint	Extension / Database state
Analogy	Like a class definition in programming	Like an object/variable in programming

Levels of Data Abstraction (Three-Level Architecture)

(Refer to Q1 above for the detailed explanation of all three levels — Physical, Logical, and View levels, as the concept is the same.)

The three levels in brief:

- Physical/Internal Level: How data is stored (file structures, indexes, storage devices)
- Logical/Conceptual Level: What data is stored and relationships among data (tables, columns, constraints)
- View/External Level: What a particular user group sees (customized views, data security)

Q: Define the terms Data Abstraction and Data Independence. Why are they important in DBMS?

Asked: 2078 Chaitra (Regular Exam); 2077 Chaitra [2+2]; 2078 Poush [3+1] | Marks: [2+3]

Answer

Data Abstraction

Data abstraction is the process of hiding the complexity of the actual physical data storage and exposing only the relevant information to users. It allows users to interact with the database without needing to know how the data is internally stored or organized.

DBMS achieves data abstraction through the three-level architecture (external, conceptual, internal levels). Each level hides certain details from the levels above it:

- Physical level hides: exact storage structures, file organization, index structures
- Conceptual level hides: physical storage details from programmers and users
- View level hides: parts of the database from certain users (security)

Importance of Data Abstraction:

- Users can work with data without knowing underlying complexity
- Physical storage can be changed without rewriting user programs
- Security is improved — users see only what they need
- Simplifies database design and application development

Data Independence

Data independence is the ability to change the schema at one level without requiring changes to the schema at the next higher level. It allows modification of the database design without affecting the applications that use it. There are two types:

a) Physical Data Independence

The ability to modify the physical schema (how data is stored) without affecting the conceptual schema or application programs. Most DBMSs support this well.

- Example: Adding a new index, changing from heap file to B+ tree, moving data to different disks
- Applications continue to work without any modification

b) Logical Data Independence

The ability to modify the conceptual schema (logical structure) without affecting external schemas (user views) or application programs. This is harder to achieve because changes to conceptual structure may break existing views.

- Example: Adding a new attribute to a table, splitting one table into two
- Existing applications continue to work through appropriate view definitions

Why are They Important in DBMS?

- Program-data independence: Application programs do not need to be rewritten when data storage changes
- Flexibility: Database administrators can tune physical storage without affecting users
- Reduced maintenance cost: Changes in one level do not propagate to other levels
- Security and privacy: Users are isolated from data they don't need to see
- Support for multiple users: Different users can have different views of the same data
- Long-term adaptability: As hardware and requirements change, the database can evolve

---

Q: Explain DDL, DML and DCL with examples.

Asked: 2081 Ashwin (Back Exam); 2070 Magh [4] | Marks: [2+4]

Answer

1. Data Definition Language (DDL)

DDL is used to define, alter, and drop database objects (tables, views, indexes, schemas). DDL statements define the structure of the database. When DDL statements are compiled, the definitions are stored in the data dictionary (a special table maintained by the DBMS).

DDL Commands with Examples:

CREATE — Creates a new database object

```
CREATE TABLE Employee (
  empid     INT          PRIMARY KEY,
  ename     VARCHAR(50)  NOT NULL,
  salary    DECIMAL(10,2),
  dept_id   INT          REFERENCES Dept(deptid)
);
```

ALTER — Modifies an existing table structure

```
ALTER TABLE Employee ADD COLUMN phone VARCHAR(15);
ALTER TABLE Employee DROP COLUMN phone;
```

DROP — Removes a database object

```
DROP TABLE Employee;
DROP INDEX idx_ename;
```

TRUNCATE — Removes all records but keeps the table structure

```
TRUNCATE TABLE Employee;
```

2. Data Manipulation Language (DML)

DML is used to retrieve and manipulate data stored in the database. SQL DML is declarative/non-procedural — users specify WHAT data they want, not HOW to get it. The DBMS query optimizer determines the best execution plan.

SELECT — Retrieves data

```
-- Find all employees with salary > 50000:
SELECT ename, salary FROM Employee WHERE salary > 50000;
```

INSERT — Adds new records

```
INSERT INTO Employee (empid, ename, salary, dept_id)
VALUES (101, 'Ram Bahadur', 45000.00, 3);
```

UPDATE — Modifies existing records

```
-- Give 10% raise to all employees in dept 3:
UPDATE Employee SET salary = salary * 1.10 WHERE dept_id = 3;
```

DELETE — Removes records

```
-- Delete employees with salary below 10000:
DELETE FROM Employee WHERE salary < 10000;
```

3. Data Control Language (DCL)

DCL is used to control access rights and privileges on database objects. It manages who can do what on which objects in the database.

GRANT — Gives privileges to users

```
-- Allow user1 to read Employee table:
GRANT SELECT ON Employee TO user1;
-- Give all privileges on Employee to admin:
GRANT ALL PRIVILEGES ON Employee TO admin;
```

REVOKE — Removes privileges from users

```
-- Remove INSERT privilege from user1:
REVOKE INSERT ON Employee FROM user1;
```

Language	Purpose	Commands	Who Uses
DDL	Define database structure (schema)	CREATE, ALTER, DROP, TRUNCATE	DBA, Database Designers
DML	Manipulate data (CRUD operations)	SELECT, INSERT, UPDATE, DELETE	Programmers, Users
DCL	Control access and permissions	GRANT, REVOKE	DBA, Security Admin

---

Q: Distinguish between a database and a DBMS. What is the advantage of separating the logical level and physical level in database design?
Asked: 2071 Magh (Back Exam) | Marks: [2+2]

Answer

Database vs DBMS

Aspect	Database	DBMS
Nature	A structured collection of related data	Software system to manage databases
Definition	Organized data representing a mini-world	Set of programs enabling data creation, retrieval, update, delete
Type	Data (passive)	Software (active)
Purpose	Stores the actual information	Provides environment to interact with stored data
Example	Student records, employee data, sales records	MySQL, Oracle, SQL Server, PostgreSQL
Analogy	Like a warehouse storing goods	Like the warehouse management system

Advantages of Separating Logical Level from Physical Level

Separating the logical (conceptual) schema from the physical (internal) schema is fundamental to the concept of data independence. This separation provides several important benefits:

- Physical independence: The DBA can change how data is physically stored (e.g., switch from heap files to indexed files, use a faster SSD, modify block sizes) without affecting logical schemas or application programs
- Portability: The same logical schema can run on different hardware and operating systems — the database can be migrated to new hardware without rewriting applications
- Performance tuning: DBAs can add or remove indexes, partition tables, reorganize storage for better performance without changing any SQL queries in the application
- Transparency: Programmers write queries based on logical tables and columns; they do not need to know how data is stored on disk
- Simplified development: Developers work with a clean, abstract view of the data rather than with raw file structures
- Maintenance: As storage technology evolves (new storage media, compression algorithms), the physical layer can be updated without touching the logical or application layer

---

Q: What difficulties would you face if you used file system directly to implement a database application? What is physical data independence?
Asked: 2071 Bhadra (Regular/Back Exam) | Marks: [3+1]

Answer

Difficulties of Using File System Directly

Using a file system directly to implement database applications leads to the following significant difficulties:

1. Data Redundancy and Inconsistency: The same data (e.g., customer address) must be maintained in multiple files, leading to duplication and potential inconsistency when updates are made to only some copies.
2. Difficulty in Accessing Data: New queries require new programs. There is no general query facility — every new data retrieval requirement must be programmed explicitly using file read/write operations.
3. Data Isolation: Data scattered in different formats across multiple files makes it difficult to combine information from different sources in a single query.
4. Integrity Problems: Integrity rules (e.g., salary > 0, foreign key constraints) must be manually coded in every application that modifies the data, making them hard to maintain and enforce consistently.
5. Atomicity Issues: If a multi-step transaction fails midway (e.g., bank transfer fails after debit but before credit), there is no automatic way to undo partial operations. The database is left in an inconsistent state.
6. Concurrent-Access Anomalies: Without built-in concurrency control, simultaneous access by multiple users can corrupt data (lost updates, dirty reads, phantom reads).
7. Security Issues: File systems provide only coarse-grained (file-level) security. Fine-grained control (e.g., one user can read salary but not SSN) is not possible.

Physical Data Independence

Physical data independence is the capacity to change the physical schema of a database (how data is stored internally — file organization, indexes, storage structures) without requiring changes to the conceptual schema, external schemas, or application programs.

Examples:

- A DBA can change the storage organization from sequential files to B+ tree indexed files without modifying any SQL queries
- Data can be moved to a new and faster storage device without changing any application code
- Indexes can be added or removed to improve query performance without affecting programs that issue SELECT queries
- Physical data independence is supported by most modern DBMS systems through the separation of physical and conceptual schemas

SUMMARY TABLE — Chapter 1 Question Coverage

The following table summarizes all Chapter 1 questions found in past TU-IOE examination papers, their weightage, and their coverage in this answer book.

No.	Question Summary	Asked In	Marks
Q1	Levels of data abstraction in RDBMS & Physical data independence	2082 Kartik	[3+1]
Q2	Why prefer DBMS over file system?	2081 Chaitra	[4]
Q3	Define database and DBMS. Advantages of DBMS over file management	2079 Chaitra	[1+3]
Q4	Define database language. Compare DBMS with file processing system	2079 Jestha	[2+3]
Q5	Why is database system more preferred over file system?	2078 Baishakh	[4]
Q6	Drawbacks of file system to store data	2072 Magh	[4]
Q7	What difficulties using file system for database? Physical data independence	2071 Bhadra	[3+1]
Q8	Distinguish database from DBMS; advantage of separating logical/physical level	2071 Magh	[2+2]
Q9	Differentiate schema and instances; disadvantages of conventional file system	2076 Baisakh	[4]
Q10	Data abstraction and data independence; why important in DBMS?	2078 Chaitra	[2+3]
Q11	Define data abstraction and data independence; importance in DBMS	2078 Chaitra (Back)	[2+3]
Q12	Layers of abstract in database design; physical data independence	2077 Chaitra	[2+2]
Q13	Schema and Instances; levels of data abstraction	2079 Ashwin	[2+3]
Q14	Explain DDL, DML and DCL with examples	2081 Ashwin	[2+4]
Q15	Difference between DDL, DML and DCL with examples	2070 Magh	[4]
Q16	Advantages of DBMS over file processing system (briefly)	2074 Bhadra	[4]

Note: Questions that address the same concept with different wording (e.g., 'Why prefer DBMS over file system' vs 'Advantages of DBMS over file processing system') are consolidated under a single comprehensive answer. Questions from different years that test identical concepts are noted in the 'Asked In' column.

— End of Chapter 1 Answer Bank —